EXPERIMENT - 3

AIM

To create an Employee–Department–Project relational database, insert at least 30 employees across 5 departments and 8 projects, and demonstrate SQL queries using **Selection, Projection, Aggregate Functions, GROUP BY, HAVING, CASE expressions, and ORDER BY**.

SOURCE CODE

```
CREATE DATABASE CompanyDB;
USE CompanyDB;
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);
CREATE TABLE Project (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(100) NOT NULL,
    budget DECIMAL(12,2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50) NOT NULL,
    salary DECIMAL(10,2),
    hire_date DATE,
    dept_id INT,
    project_id INT,
    manager_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id),
    FOREIGN KEY (project_id) REFERENCES Project(project_id),
    FOREIGN KEY (manager_id) REFERENCES Employee(emp_id)
);
```

-- Insert Departments

INSERT INTO Department VALUES

(1,'Development'),

(2,'PeopleOps'),

(3,'Accounts'),

(4,'Sales'),

(5,'Logistics');

-- Insert Projects

INSERT INTO Project VALUES

(101,'Web Platform',150000,1),

(102,'Data Insights',200000,1),

(103,'Talent Hub',80000,2),

(104,'Audit Dashboard',120000,3),

(105,'Sales Campaign',90000,4),

(106,'Logistics Tracker',180000,5),

(107,'Mobile Suite',140000,1),

(108,'Employee Care',60000,2);

-- Insert 30 Employees

INSERT INTO Employee VALUES

(1,'Ishaan',65000,'2022-01-10',1,101,NULL),

(2,'Reyansh',72000,'2021-03-15',1,102,1),

(3,'Adit',58000,'2023-06-20',1,107,2),

(4,'Aarush',81000,'2020-08-12',1,101,3),

(5,'Raghav',69000,'2022-11-05',1,102,4),

(6,'Vihaan',62000,'2024-02-18',1,107,5),

(7,'Anika',52000,'2022-04-11',2,103,NULL),

(8,'Kavya',48000,'2023-01-22',2,108,7),

(9,'Ira',61000,'2021-07-19',2,103,8),

(10,'Naina',55000,'2022-09-30',2,108,9),

(11,'Aditi',47000,'2024-03-14',2,103,10),

(12,'Saanvi',59000,'2020-12-01',2,108,11),

(13,'Harsh',75000,'2021-02-17',3,104,NULL),

(14,'Mohit',68000,'2022-05-09',3,104,13),

(15,'Ritvik',83000,'2019-10-21',3,104,14),

(16,'Ayush',62000,'2023-08-13',3,104,15),

(17,'Kunal',71000,'2021-11-28',3,104,16),

(18,'Varun',56000,'2024-01-09',3,104,17),

(19,'Mehak',54000,'2022-02-25',4,105,NULL),

(20,'Pihu',63000,'2021-06-16',4,105,19),

(21,'Shreya',57000,'2023-04-10',4,105,20),

(22,'Tara',66000,'2020-09-07',4,105,21),

(23,'Mahi',51000,'2024-02-05',4,105,22),

(24,'Anvi',70000,'2022-12-19',4,105,23),

(25,'Rajat',60000,'2021-01-12',5,106,NULL),

(26,'Yuvraj',73000,'2020-04-23',5,106,25),

(27,'Arnav',67000,'2022-07-18',5,106,26),

(28,'Laksh',59000,'2023-05-29',5,106,27),

(29,'Devansh',76000,'2019-08-31',5,106,28),

(30,'Om',64000,'2024-01-20',5,106,29);

SQL QUERIES

1. Selection

Display employees whose salary is greater than 70,000.

```
SELECT *
FROM Employee
WHERE salary > 70000;
```

OUTPUT

KUMAR SHAILENDRA

I/O | AI Agent | 108 ms

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	salary	hire_date	dept_id	project_id	manager_id
2	Reyansh	72000.00	2021-03-15	1	102	1
4	Aarush	81000.00	2020-08-12	1	101	3
13	Harsh	75000.00	2021-02-17	3	104	NULL
15	Ritvik	83000.00	2019-10-21	3	104	14
17	Kunal	71000.00	2021-11-28	3	104	16
26	Yuvraj	73000.00	2020-04-23	5	106	25
29	Devansh	76000.00	2019-08-31	5	106	28

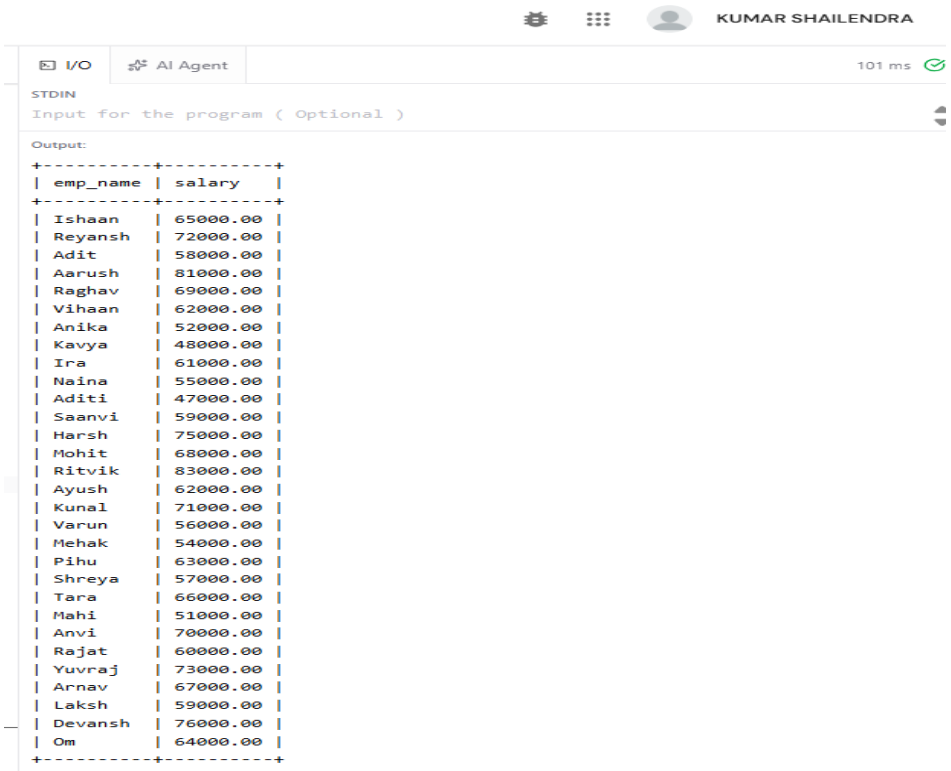
2. Projection

Display only employee names and salaries.

SELECT emp_name, salary

FROM Employee;

OUTPUT



3. Aggregate Functions

Find total employees, average salary, maximum salary, minimum salary and total salary.

```
SELECT
    COUNT(*) AS total_employees,
    AVG(salary) AS average_salary,
    MAX(salary) AS highest_salary,
    MIN(salary) AS lowest_salary,
    SUM(salary) AS total_salary
FROM Employee;
```

Output

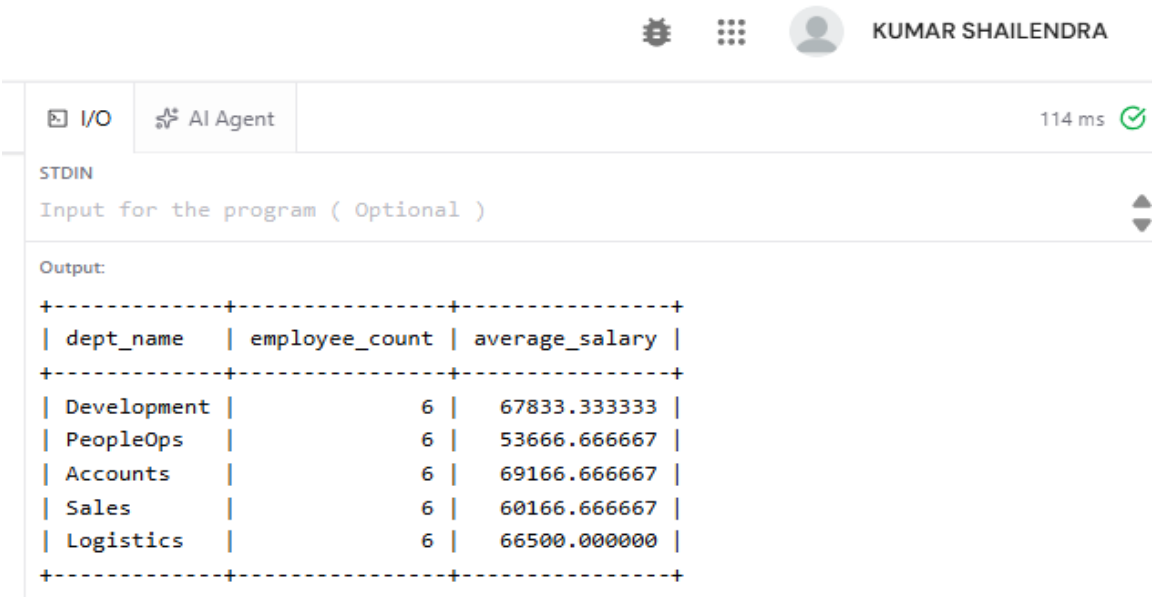


4. GROUP BY

Display the number of employees and average salary in each department.

```
SELECT
    d.dept_name,
    COUNT(e.emp_id) AS employee_count,
    AVG(e.salary) AS average_salary
FROM Department d
JOIN Employee e
ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

OUTPUT

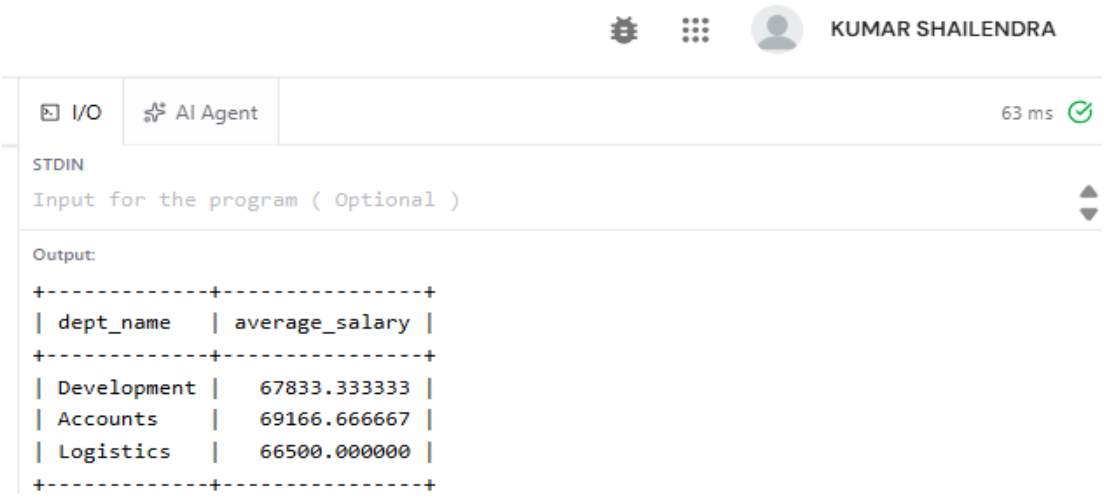


5. HAVING

Display departments whose average salary is greater than 65,000.

```
SELECT
   d.dept_name,
   AVG(e.salary) AS average_salary
FROM Department d
JOIN Employee e
ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) > 65000;
```

OUTPUT

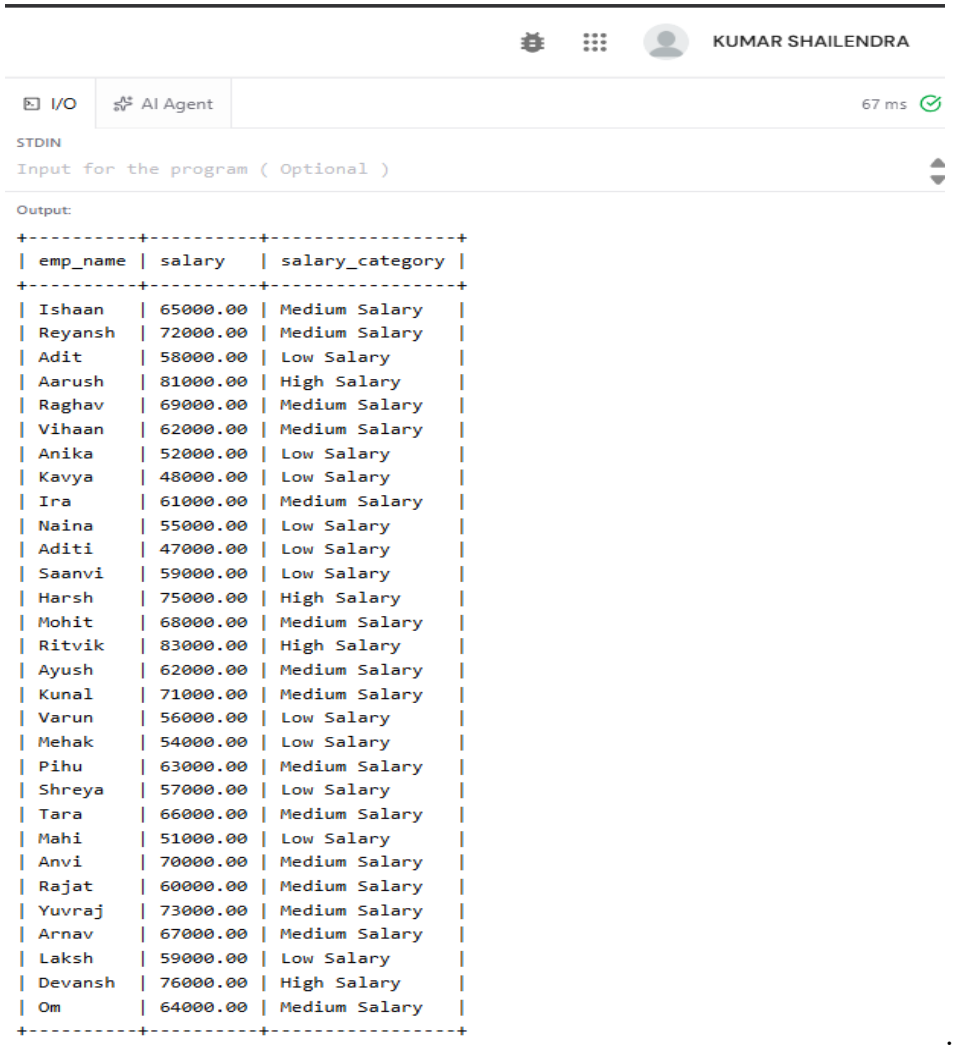


6. CASE Expression

Classify employees according to their salary.

```
SELECT
   emp_name,
   salary,
   CASE
      WHEN salary >= 75000 THEN 'High Salary'
      WHEN salary >= 60000 THEN 'Medium Salary'
      ELSE 'Low Salary'
   END AS salary_category
FROM Employee;
```

OUTPUT




```
+----------+----------+-----------------+
| emp_name | salary   | salary_category |
+----------+----------+-----------------+
| Ishaan   | 65000.00 | Medium Salary   |
| Reyansh  | 72000.00 | Medium Salary   |
| Adit     | 58000.00 | Low Salary      |
| Aarush   | 81000.00 | High Salary     |
| Raghav   | 69000.00 | Medium Salary   |
| Vihaan   | 62000.00 | Medium Salary   |
| Anika    | 52000.00 | Low Salary      |
| Kavya    | 48000.00 | Low Salary      |
| Ira      | 61000.00 | Medium Salary   |
| Naina    | 55000.00 | Low Salary      |
| Aditi    | 47000.00 | Low Salary      |
| Saanvi   | 59000.00 | Low Salary      |
| Harsh    | 75000.00 | High Salary     |
| Mohit    | 68000.00 | Medium Salary   |
| Ritvik   | 83000.00 | High Salary     |
| Ayush    | 62000.00 | Medium Salary   |
| Kunal    | 71000.00 | Medium Salary   |
| Varun    | 56000.00 | Low Salary      |
| Mehak    | 54000.00 | Low Salary      |
| Pihu     | 63000.00 | Medium Salary   |
| Shreya   | 57000.00 | Low Salary      |
| Tara     | 66000.00 | Medium Salary   |
| Mahi     | 51000.00 | Low Salary      |
| Anvi     | 70000.00 | Medium Salary   |
| Rajat    | 60000.00 | Medium Salary   |
| Yuvraj   | 73000.00 | Medium Salary   |
| Arnav    | 67000.00 | Medium Salary   |
| Laksh    | 59000.00 | Low Salary      |
| Devansh  | 76000.00 | High Salary     |
| Om       | 64000.00 | Medium Salary   |
+----------+----------+-----------------+
```

...

7. ORDER BY

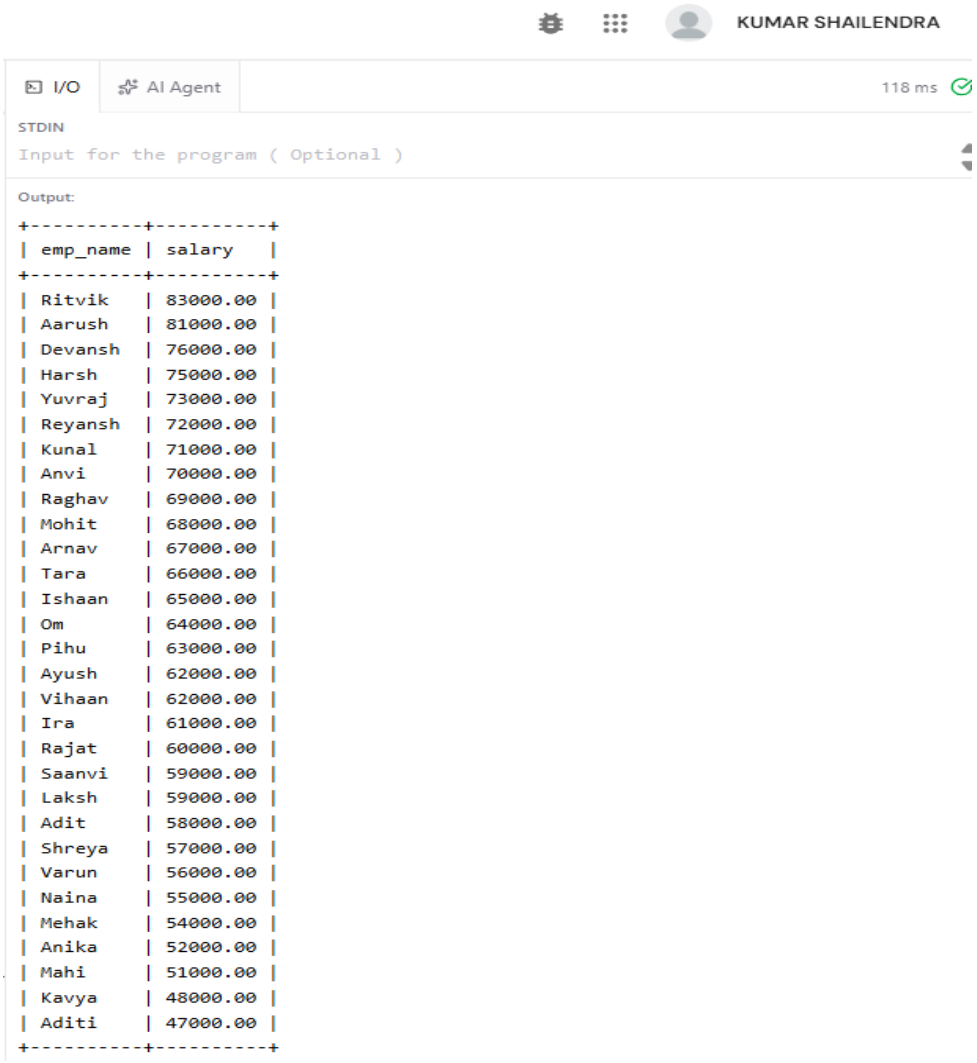
Display employees in descending order of salary.

SELECT emp_name, salary

FROM Employee

ORDER BY salary DESC;

OUTPUT

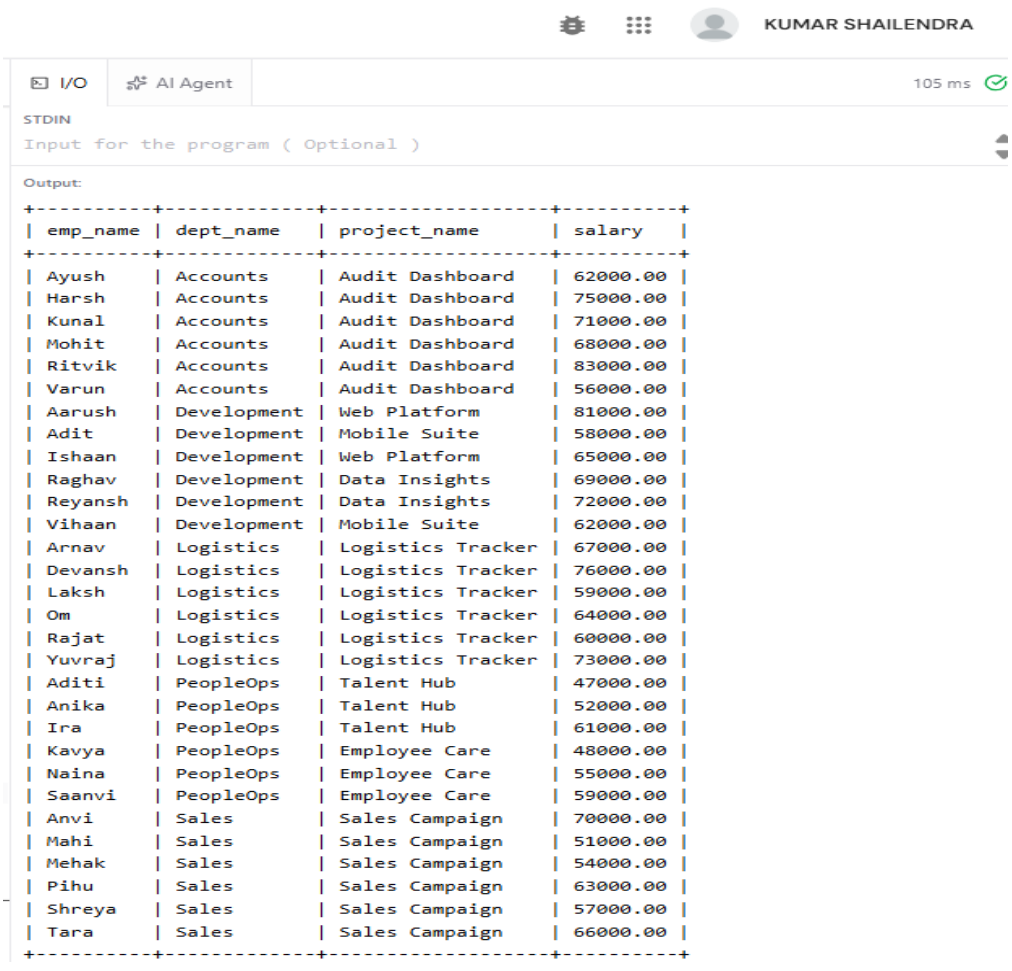


8. JOIN Employee, Department and Project

Display employee name, department, project and salary.

```
SELECT
    e.emp_name,
    d.dept_name,
    p.project_name,
    e.salary
FROM Employee e
JOIN Department d
ON e.dept_id = d.dept_id
JOIN Project p
ON e.project_id = p.project_id
ORDER BY d.dept_name, e.emp_name;
```

OUTPUT



emp_name	dept_name	project_name	salary
Ayush	Accounts	Audit Dashboard	62000.00
Harsh	Accounts	Audit Dashboard	75000.00
Kunal	Accounts	Audit Dashboard	71000.00
Mohit	Accounts	Audit Dashboard	68000.00
Ritvik	Accounts	Audit Dashboard	83000.00
Varun	Accounts	Audit Dashboard	56000.00
Aarush	Development	Web Platform	81000.00
Adit	Development	Mobile Suite	58000.00
Ishaan	Development	Web Platform	65000.00
Raghav	Development	Data Insights	69000.00
Reyansh	Development	Data Insights	72000.00
Vihaan	Development	Mobile Suite	62000.00
Arnav	Logistics	Logistics Tracker	67000.00
Devansh	Logistics	Logistics Tracker	76000.00
Laksh	Logistics	Logistics Tracker	59000.00
Om	Logistics	Logistics Tracker	64000.00
Rajat	Logistics	Logistics Tracker	60000.00
Yuvraj	Logistics	Logistics Tracker	73000.00
Aditi	PeopleOps	Talent Hub	47000.00
Anika	PeopleOps	Talent Hub	52000.00
Ira	PeopleOps	Talent Hub	61000.00
Kavya	PeopleOps	Employee Care	48000.00
Naina	PeopleOps	Employee Care	55000.00
Saanvi	PeopleOps	Employee Care	59000.00
Anvi	Sales	Sales Campaign	70000.00
Mahi	Sales	Sales Campaign	51000.00
Mehak	Sales	Sales Campaign	54000.00
Pihu	Sales	Sales Campaign	63000.00
Shreya	Sales	Sales Campaign	57000.00
Tara	Sales	Sales Campaign	66000.00

RESULT

The **Employee–Department–Project database** was successfully created with:

- 30 Employees
- 5 Departments
- 8 Projects

The following SQL concepts were successfully demonstrated:

- Selection
- Projection
- Aggregate Functions
- GROUP BY
- HAVING
- CASE Expression
- ORDER BY
- JOIN